COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	7/16/26	**DATE SUBMITTED:**	7/16/26
NAME:	Busa, Emmanuel A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 **HOA Arrays, pointers and dynamicemory allocation**

1. PROCEDURE OUTPUT

ILO A

Screenshot

1st.cpp | modified.cpp | Final.cpp

```cpp
#include <iostream>
#include <string.h>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    Student(std::string newName = "John Doe", int newAge = 18) {
        studentName = newName;
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    };


    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }

    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called." << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    void printDetails() {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

int main() {
    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}
```

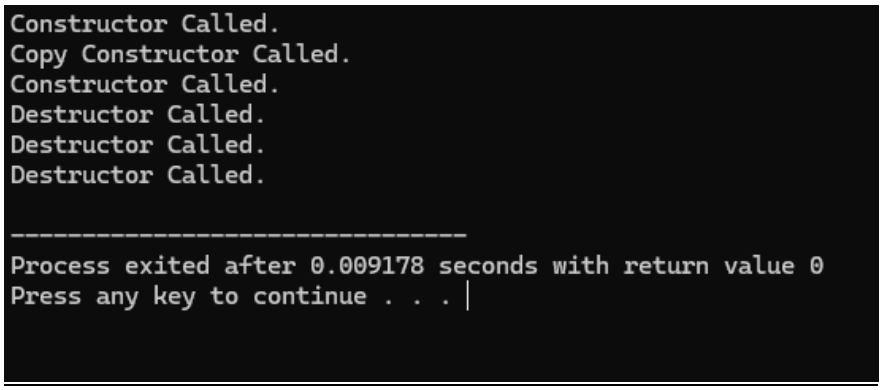
	
Observation	The student1 was created, so the constructor is called. Then student2 was initialized using student1 so it triggered the copy constructor because it was copying student1. student3 is created using a constructor. When I assigned student2 to student3, no copy constructor was called because the object already existed. The destructors are called automatically in the end in reverse order of object creation.

Table 2-1. Initial Driver Program

Screenshot

1st.cpp | modified.cpp | Final.cpp

```cpp
#include <iostream>
#include <string.h>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    Student(std::string newName = "John Doe", int newAge = 18) {
        studentName = newName;
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    };


    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }

    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called." << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    void printDetails() {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};


int main() {
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    for (int i = 0; i < j; i++) {
        Student *ptr = new Student(namesList[i], ageList[i]);
        studentList[i] = *ptr;
    }

    for (int i = 0; i < j; i++) {
        studentList[i].printDetails();
    }

    return 0;
}
```

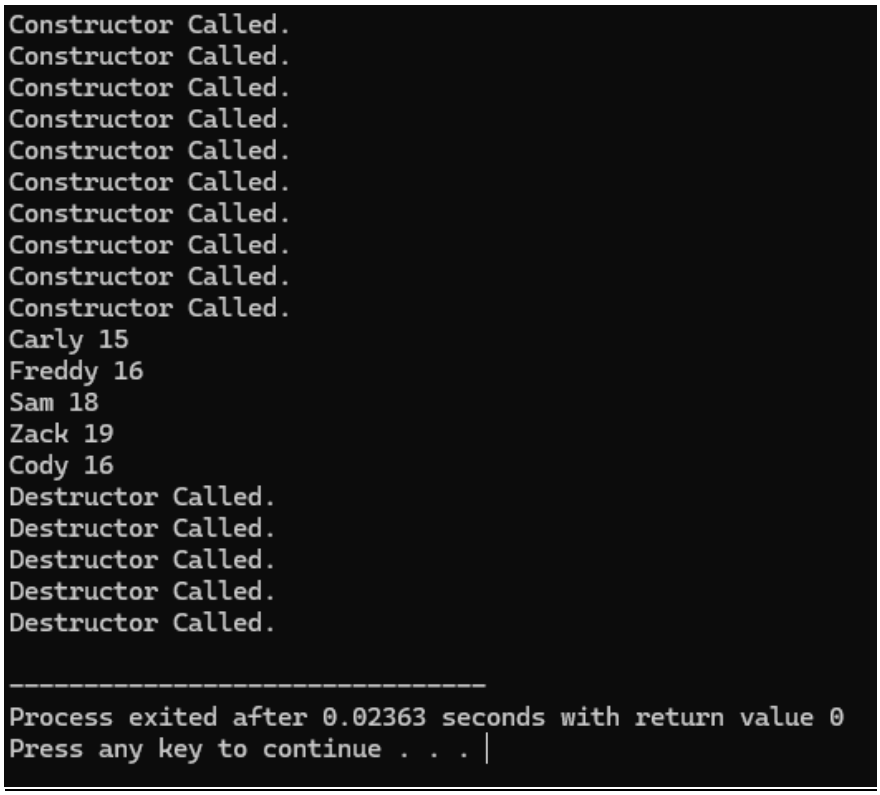
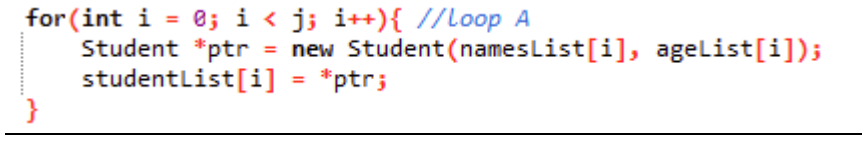
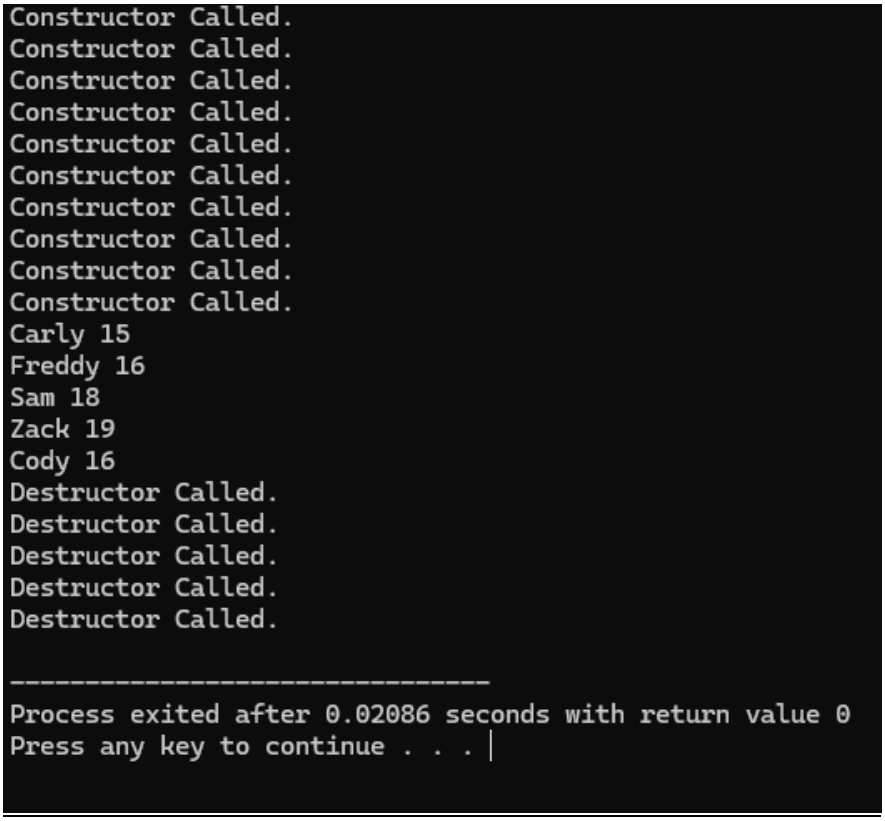
	
Observation	From this modification, I learned that constructors initialize new objects, copy constructors duplicate existing object data, assignment changes the values of an already created object, and destructors handle object cleanup automatically.

Table 2-2. Modified Driver Program with Student Lists

Loop A	
Observation	In Loop A, I observed that the program uses a loop to create and assign values to each Student in the studentList array. Each iteration creates a new Student object using the names and ages from namesList and ageList which calls the

	constructor. The created student information is copied into the corresponding index of studentList allowing the array to store all student details.

Loop B	`for(int i = 0; i < j; i++){ //loop B`
	`  studentList[i].printDetails();`
	`}`

Observation	In Loop B, I observed that the program uses a loop to access each object stored in the studentList array. The printDetails() function is called for every student object to display their name and age. This shows how loops can be used to efficiently retrieve and display data from multiple objects in C++
Output	
Observation	When I ran the program, I observed that the constructor was called multiple times before any student information was displayed. This indicates that several objects were created at the start of the program. After the objects were initialized, the program displayed the names and ages of five students: Carly, Freddy, Sam, Zack, and Cody. After the output was displayed, I noticed that the destructor was called five times. This shows that the objects were properly destroyed when they went out of scope or when the program terminated. The output demonstrates the order of object creation and destruction in C++, where constructors are executed when objects are created and destructors are executed when objects are destroyed. Through the output, I was able to visualize the object lifecycle in C++, starting from object instantiation, data display, and finally object destruction at the

	end of program execution

Table 2-3. Final Driver Program

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Chatgpt

Purpose of Use:

idea generation, explanation of concepts

Extent of Use:

it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.